

Phase 4

Project Planning Phase

Epics, tasks, milestones, tools and risks

1. Planning Approach

The project is organized into five epics, executed in sequence: architecture first, then setup, backend development, database configuration and finally execution and testing. Each epic produces concrete deliverables that feed the next.

2. Epic Breakdown

Epic	Scope	Key deliverables	Status
Epic 1: Project Architecture	Design the overall system	Architecture diagram, components definition, ER diagram, user flow diagram, MVC pattern	Completed
Epic 2: Project Setup and Configuration	Prepare the development environment	Project folder, npm init, dependencies, Nodemon, base files (index.js, .env, package.json, .gitignore), folder structure, environment variables	Completed
Epic 3: Backend Development	Build core backend modules	Entry point (index.js), database connection, authentication middleware, Gemini AI integration, file upload middleware	Completed
Epic 4: Database Configuration	Set up MongoDB and schemas	Mongoose connection, User schema, Study Material schema, collections, API route endpoints (Auth, Material, AI, Admin)	Completed
Epic 5: Project Execution	Run and verify the system	Run the server, test APIs in Postman (register, login, upload, summarize, flashcards, quiz, study plan), demo videos	Completed

3. Task Plan per Epic

Epic 2: Setup and Configuration

- Create folder named AI StudyBuddy and open it in Visual Studio Code.
- Initialize with `npm init -y`.
- Install: express, mongoose, bcryptjs, jsonwebtoken, cors, dotenv, multer and the Google GenAI package; install nodemon as a dev dependency.
- Create index.js, .env, package.json, .gitignore and the src folders (controllers, middleware, models, routes, utils).
- Configure PORT, MONGO_URI, JWT secret and GEMINI_API_KEY in .env.

Epic 3 and 4: Development and Database

- Implement server entry point with middleware, routes, health check and global error handler.
- Connect to MongoDB before starting the server.
- Implement JWT protect and adminOnly middleware; Multer upload middleware.
- Implement askGemini utility for all AI prompts.
- Define User and Material schemas and expose Auth, Material, AI and Admin APIs.

Epic 5: Execution

- Install dependencies with `npm install` and start with `npm start`.
- Verify each API in Postman and record the backend development and API execution demos.

4. Milestones

Milestone	Completed when
M1: Design approved	Architecture, ER and user-flow diagrams and MVC structure are defined
M2: Environment ready	Dependencies installed, folder structure and .env configured
M3: Core backend working	Server starts, connects to MongoDB, auth and upload middleware in place
M4: AI features integrated	Gemini utility generates summary, flashcards, quiz and study plan
M5: Verified and demonstrated	Postman tests pass and demo videos are recorded

5. Tools and Resources

- Visual Studio Code, Node.js, npm, Nodemon
- MongoDB and Mongoose
- Postman / Thunder Client for API testing
- Google Gemini API key
- Git (.gitignore configured to keep secrets out of the repository)

6. Risks and Mitigation

Risk	Mitigation built into the design
Unauthorized access to AI features and data	JWT authentication, protect middleware and role-based adminOnly check
Password exposure	Passwords hashed with bcrypt before saving
Invalid or oversized uploads	Multer file filter (.txt, .md, .pdf) and 5 MB limit
Secret / API key leakage	Secrets stored in .env, excluded through .gitignore
Slow AI responses (observed 5 to 20 seconds in testing)	AI logic isolated in the gemini.js utility; clients should show a loading state while waiting
Tight coupling to one AI provider	Single askGemini function makes the AI layer easy to change

Note: the source document does not list calendar dates or team assignments, so this plan is organized by epics and milestones taken from the document rather than by dates.